

linear_perceptron.py

Perceptrón Lineal (Adaline) — Código Fuente

TP3 — Sistemas de Inteligencia Artificial — ITBA 2026

Imports y dependencias

```
import argparse
import os
from datetime import datetime

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

train_perceptron — Entrenamiento del perceptrón (Adaline, online learning)

```
def train_perceptron(df, learning_rate=0.01, epochs=1000, epsilon=1e-4):
    # Extract data
    x_raw = df["x"].values
    z = df["y"].values          # salida esperada ( $z^{\mu}$ )
    P = len(z)

    # Prepend  $x_0 = 1$  for bias trick -> X shape: (P, 2)
    X = np.column_stack([np.ones(P), x_raw])

    # Initialize weights with small random values
    rng = np.random.default_rng()
    weights = rng.uniform(-0.1, 0.1, size=2)

    mse_history = []

    for epoch in range(epochs):
        # Online learning: iterate over each data point
        for mu in range(P):
            # Excitation - identity activation:  $O_{\mu} = h_{\mu}$ 
            O_mu = np.dot(weights, X[mu])

            # Update:  $w_i += \eta * (z_{\mu} - O_{\mu}) * x_{i_{\mu}}$ 
            error = z[mu] - O_mu
            weights = weights + learning_rate * error * X[mu]

        # MSE over entire dataset
        predictions = X @ weights
        mse = np.mean((z - predictions) ** 2)
        mse_history.append(mse)
```

```

        if mse < epsilon:
            break

    return weights, mse_history

```

run_and_save — Pipeline: entrenar, guardar pesos y graficar

```

def run_and_save(csv_path, learning_rate, epochs, epsilon, output_dir):
    df = pd.read_csv(csv_path)
    weights, mse_history = train_perceptron(
        df, learning_rate, epochs, epsilon
    )

    os.makedirs(output_dir, exist_ok=True)

    # Save weights
    wdf = pd.DataFrame({
        "w0": [weights[0]], "w1": [weights[1]],
        "mse": [mse_history[-1]]
    })
    wdf.to_csv(os.path.join(output_dir, "weights.csv"), index=False)

    # Plot
    x = df["x"].values
    y = df["y"].values
    x_line = np.linspace(x.min(), x.max(), 100)
    y_line = weights[0] + weights[1] * x_line

    plt.figure(figsize=(10, 6))
    plt.scatter(x, y, label="Datos", alpha=0.7)
    plt.plot(x_line, y_line, "r-", linewidth=2,
             label=f"Perceptron: y = {weights[1]:.3f}x "
                  f"+ {weights[0]:.3f}")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title(f"Perceptron Lineal -- MSE final: "
              f"{mse_history[-1]:.6f} ({len(mse_history)} epocas)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.savefig(os.path.join(output_dir, "plot.png"),
                dpi=150, bbox_inches="tight")
    plt.close()

    print(f"Weights: w0={weights[0]:.4f}, w1={weights[1]:.4f}")
    print(f"MSE final: {mse_history[-1]:.6f} "
          f"({len(mse_history)} epochs)")
    print(f"Output saved to {output_dir}")

```

main — CLI con argparse

```

def main():

```

```
parser = argparse.ArgumentParser(
    description="Linear Perceptron (Adaline)"
)
parser.add_argument("--csv", required=True,
    help="Path to input CSV with columns x, y")
parser.add_argument("--learning_rate", type=float, default=0.01,
    help="Learning rate (default: 0.01)")
parser.add_argument("--epochs", type=int, default=1000,
    help="Max epochs (default: 1000)")
parser.add_argument("--epsilon", type=float, default=1e-4,
    help="MSE convergence threshold (default: 1e-4)")
args = parser.parse_args()

csv_basename = os.path.splitext(os.path.basename(args.csv))[0]
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_dir = f"output_{csv_basename}_{timestamp}"

run_and_save(
    args.csv, args.learning_rate,
    args.epochs, args.epsilon, output_dir
)

if __name__ == "__main__":
    main()
```